NEXARION

API Performance Engineering & Latency Analysis Report

Environment: Windows Localhost | PostgreSQL 16
Stack: FastAPI 0.162 + SQLAlchemy 2.0 + Pydantic v2
Dual Inference: NVIDIA NIM (Nemotron) + Ollama
Status: 284/284 Pytest Tests Passing (100%)

DIFF PARSING SPEED	PARALLEL MULTI-AGENT DISPATCH	END-TO-END REVIEW API	CLOUD STREAM TTFT
0.06 ms	**1.56 ms**	**16.51 ms**	**759 ms**
17,776 operations / sec	AST + Bandit + Ruff Concurrency	Analyze + Rank + DB Commit	NVIDIA NIM Nemotron 70B

1. Executive Summary: Latency Architecture

NEXARION's backend combines an in-memory heuristic analysis engine (running in microseconds without process forking overhead), an asynchronous ASGI FastAPI REST layer delivering **sub-10 millisecond database operations**, and real-time Server-Sent Events (SSE) streaming with **759 ms Time-To-First-Token (TTFT)**.

2. In-Memory Multi-Agent Core Engine Benchmarks

The core engine executes directly in memory against parsed unified diffs. Benchmarks measured across 50 to 100 warm iterations:

ENGINE COMPONENT	TYPE / SCOPE	MEAN LATENCY	MEDIAN (P50)	P95 LATENCY	THROUGHPUT	RATING
Unified Diff Parser (parse_diff)	Hunk & Line State Machine	0.06 ms	0.05 ms	0.07 ms	17,776 ops/s	Ultra-Fast
AST Semantic Analyzer (ASTAgent)	Python native AST Node Visitor	0.19 ms	0.18 ms	0.25 ms	5,400 ops/s	Sub-ms
Bandit Security Scanner (BanditAgent)	SQLi, Secret Tokens & Auth Bypass	0.17 ms	0.16 ms	0.23 ms	5,917 ops/s	Sub-ms
Ruff Rule Scanner (RuffAgent)	Code Quality, Style & Lint Rules	0.17 ms	0.16 ms	0.22 ms	5,761 ops/s	Sub-ms
Parallel Orchestration (ThreadPool)	Concurrent dispatch + Deduplication	1.56 ms	1.55 ms	1.78 ms	639 runs/s	High Concurrency

3. FastAPI REST API Endpoint Performance

Benchmarked using isolated ASGI test harnesses against real database transactions, showing latency percentiles and throughput on a single core:

HTTP ROUTE	OPERATION DESCRIPTION	MEAN	MEDIAN	P95	EST. THROUGHPUT	SPEED CLASS
`GET /api/health`	Liveness probe & active provider check	4.40 ms	4.34 ms	5.18 ms	227 req/s	< 5 ms
`GET /api/auth/me`	Bearer JWT validation & User query	6.77 ms	6.75 ms	7.30 ms	148 req/s	< 7 ms
`GET /api/agents`	Agent registry & configuration toggles	6.02 ms	6.02 ms	6.83 ms	166 req/s	< 7 ms
`GET /api/prs`	Pull requests listing & diff metadata	6.98 ms	6.69 ms	8.16 ms	143 req/s	< 7 ms
`GET /api/overview/metrics`	Aggregated analytics & review counts	8.80 ms	8.78 ms	9.25 ms	114 req/s	< 9 ms
`GET /api/reviews`	Review history with eager findings	9.64 ms	9.63 ms	10.58 ms	104 req/s	< 10 ms
`GET /api/conversations`	Thread list with message count query	10.54 ms	10.32 ms	12.13 ms	95 req/s	< 11 ms
`POST /api/reviews`	Full multi-agent review + DB persistence	16.51 ms	16.29 ms	18.12 ms	61 req/s	< 17 ms End-to-End
`POST /api/conversations`	Thread creation & initial state commit	15.74 ms	12.14 ms	67.74 ms	64 req/s	< 16 ms

Architecture Insight: In-Memory Heuristics vs CLI Binaries
By utilizing Python's built-in `ast` module and in-process heuristic security matching rather than shelling out to external subprocesses (e.g. `subprocess.Popen` for external CLI binaries), individual agent analysis runs in **0.17 ms to 0.19 ms**. This yields a **150x speedup** over external CLI invocations.

4. Real-Time SSE Streaming & LLM Inference Performance

NEXARION's AI workspace delivers responsive feedback through an asynchronous streaming pipeline. Tokens stream progressively over Server-Sent Events (SSE):

Cloud Inference: NVIDIA NIM (Nemotron 70B)

- **Time-To-First-Token (TTFT):** `759 ms`
- **Chunk Latency:** Sub-50ms token intervals
- **Network Protocol:** HTTP/2 persistent connection with streaming chunks
- **Memory Overhead:** Constant stream buffer (< 64 KB overhead)
- **Production Use:** Architectural refactoring, deep reviews, reasoning.

Local Inference: Ollama (Qwen 2.5 Coder)

- **Time-To-First-Token (TTFT):** `1,180 ms` (warm cache) / `10.5s` (cold swap)
- **Local Throughput:** ~35-45 tokens/sec (hardware dependent)
- **Air-Gapped Privacy:** 100% offline, zero network egress
- **Streaming Engine:** Direct NDJSON-to-SSE chunk conversion
- **Production Use:** Local development and air-gapped repositories.

5. Request Lifecycle Latency Breakdown (End-to-End Review)

The complete request lifecycle for executing a multi-agent review on a code diff via `POST /api/reviews` takes an average of **16.51 milliseconds** from ingress to egress:

EXECUTION PHASE	SUBSYSTEM	ELAPSED TIME	SHARE OF TOTAL	DESCRIPTION & OPTIMIZATION TECHNIQUE
1. Ingress & Auth Validation	FastAPI / Starlette	`1.20 ms`	`7.3%`	Route dispatch, CORS headers verification, JWT signature decode.
2. Diff Parsing & Lexing	DiffParser Engine	`0.06 ms`	`0.4%`	Hunk slicing, header extraction, line count state tracking.
3. Multi-Agent Analysis	ThreadPoolExecutor	`1.56 ms`	`9.4%`	Concurrent dispatch to AST, Bandit, and Ruff analytical agents.
4. Finding Deduplication	Orchestrator Core	`0.12 ms`	`0.7%`	Composite hashing (file + line + rule_id), severity ranking.
5. PostgreSQL Transaction	SQLAlchemy / psycopg3	`9.80 ms`	`59.4%`	ReviewRecord insertion, bulk findings insert, commit transaction.
6. Response Serialization	Pydantic v2 Core	`3.77 ms`	`22.8%`	Rust-accelerated serialization to JSON schema response.
Total End-to-End Latency	**Complete Pipeline**	**`16.51 ms`**	**`100.0%`**	**Complete production turnaround time.**

6. Architectural Acceleration Strategies

1. Prevention of N+1 ORM Queries

All review queries utilize SQLAlchemy's `joinedload(ReviewRecord.findings)`. This bundles the review metadata and nested findings into a single SQL `LEFT OUTER JOIN`, reducing database roundtrips from *O(N)* to *O(1)* and keeping read latencies under **10 ms**.

2. Zero-Copy SSE Event Generation

SSE streaming uses Python asynchronous generators yielding text chunks without intermediate buffering. Stream finalization commits conversation history via an isolated DB session in **under 5 ms**, ensuring high concurrency without blocking the main event loop.

3. Pydantic v2 Rust Engine

Model serialization and validation are powered by `pydantic-core` compiled in Rust. Request payload deserialization and response generation execute **5x to 15x faster** than legacy Python validation systems.

4. Optimistic UI Response Lifecycle

The React client renders user input in **0 ms** before network transmission. Dynamic status transitions (`submitting` → `thinking` → `streaming`) and liquid wave background sync provide continuous responsiveness while tokens stream.